오픈소스SW의 이해

# 응용 모의퀴즈

# Git·명령어·브랜치

CHAPTER 1 실습 + CHAPTER 3 실습

## 75문항 · 15문항 × 5회

문제 15개를 모두 푼 뒤, 이어지는 정답·해설로 채점하세요. 각 회차는 문제 → 정답 순서입니다.


1회 VCS·워크플로·터미널

문제 2쪽 / 정답 7쪽 점수 _____ / 30

2회 작업트리·스테이지·커밋

문제 12쪽 / 정답 17쪽 점수 _____ / 30

3회 브랜치 포인터와 이력 범위

문제 22쪽 / 정답 27쪽 점수 _____ / 30

4회 병합 방향·충돌·빨리감기

문제 32쪽 / 정답 37쪽 점수 _____ / 30

5회 복합 상태 추적·역추론

문제 42쪽 / 정답 47쪽 점수 _____ / 30


## 자가채점 안내

문항당 2점, 총 150점입니다. 객관식·복수선택·순서·집합형은 전체 일치 시 2점입니다. OX형은 판정 1점 + 근거·수정 1점이며, 나머지는 정답에 표시한 배점으로 채점하세요. 뜻이 같은 표현과 동등한 명령도 인정합니다.

이름 _____ 총점 _____ / 150

# 01회 문제

VCS·워크플로·터미널 | 1 / 5

## 01 | 객관식 · 2점

이미지 디자이너가 수정 이력과 이전 버전을 관리하려 한다. VCS에 대한 옳은 설명은?
① 소스코드만 관리 가능
② 이미지 등 컴퓨터 파일도 관리 가능
③ GitHub 계정이 없으면 불가
④ 파일 이름을 매번 바꿔야 가능

답 / 풀이

## 02 | 조건비교 · 2점

중앙 서버가 멈췄다. CVCS와 완전한 Git 복제본을 가진 DVCS에서 각각 가능한 작업을 비교하라. 인터넷 공유는 불가능한 상황이다.

답 / 풀이

## 03 | OX·수정 · 2점

“Git을 중앙 저장소 하나를 중심으로 운영하면 Git 자체가 CVCS로 바뀐다.” O/X와 근거를 쓰시오.

답 / 풀이

# 01회 문제

VCS·워크플로·터미널 | 2 / 5

## 04 | 연결형 · 2점

A: 기여자가 자기 저장소에 올리고 대표자에게 병합 요청. B: 부분 관리자들이 취합하고 최종 관리자가 반영. 워크플로 이름을 연결하라.

답 / 풀이

## 05 | 객관식 · 2점

Git 개발 목표와 가장 거리가 먼 것은?
① 비선형 개발 지원
② 대형 프로젝트 처리
③ 빠른 속도
④ 모든 로컬 커밋에 중앙 서버 승인 필수

답 / 풀이

## 06 | 명령작성 · 2점

현재 위치가 /home/so/project/src 이다. 상위 폴더로 이동한 뒤 절대경로를 확인하는 명령 두 개와 출력 경로를 쓰시오.

답 / 풀이

# 01회 문제

VCS·워크플로·터미널 | 3 / 5

## 07 | 결과추적 · 2점

처음 위치는 /home/so/A이다. 다음 명령을 차례로 실행했다.
cd /home/so/B
cd -
pwd
마지막 경로와 cd -의 의미를 쓰시오.

답 / 풀이

## 08 | 명령작성 · 2점

현재 폴더 아래 exam/git/practice를 한 번에 만들려 한다. 중간 폴더 exam과 git은 없다. 필요한 명령과 옵션 의미를 쓰시오.

답 / 풀이

## 09 | 객관식 · 2점

.git을 포함한 숨김 항목, 상세 정보, 읽기 쉬운 크기를 한꺼번에 보려 한다. 알맞은 것은?
① ls -lah
② pwd -a
③ clear -l
④ cd -h

답 / 풀이

# 01회 문제

VCS·워크플로·터미널 | 4 / 5

## 10 | OX·수정 · 2점

“clear를 실행하면 작업 폴더 파일과 Git 커밋 이력이 삭제된다.” O/X와 실제 효과를 쓰시오.

답 / 풀이

## 11 | 오류수정 · 2점

비어 있지 않은 임시 폴더 temp를 삭제하려고 rm temp를 썼다. 일반적인 디렉토리 삭제 옵션을 넣은 명령과 삭제 범위를 쓰시오. temp만 삭제 대상으로 가정한다.

답 / 풀이

## 12 | 빈칸 · 2점

Bash에서 홈 디렉토리로 이동하는 명령은 (A), 터미널을 종료하는 명령은 (B)이다.

답 / 풀이

# 01회 문제

VCS·워크플로·터미널 | 5 / 5

## 13 | 조건추론 · 2점

Git 저장소에서 일반 ls로 .git이 안 보였다. “버전관리가 사라졌다”는 결론을 내리기 전에 확인할 명령과 이유를 쓰시오.

답 / 풀이

## 14 | 명령작성 · 2점

현재 사용자 계정의 여러 저장소에 기본 작성자 이름을 SSW로 설정하려 한다. Git 설정 명령과 적용 범위를 쓰시오.

답 / 풀이

## 15 | 통합판단 · 2점

서버가 고장났지만 한 팀원이 프로젝트의 전체 Git 이력을 복제해 갖고 있다. 복구 가능성의 근거와 복제본에 없을 수 있는 자료 한 가지를 쓰시오.

답 / 풀이

# 01회 정답·해설

VCS·워크플로·터미널 | 1 / 5

## 01 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

②

해설

버전관리는 파일의 시간에 따른 변화를 기록한다. 코드만의 기능이 아니다.

근거 실습1 p.6

## 02 | 조건비교 · 내 점수 ___ / 2

정답 · 채점 기준

CVCS: 서버 의존 이력 조회·커밋 등 제약 (1점). DVCS: 로컬 이력 조회·커밋 등 계속 가능 (1점).

해설

CVCS도 로컬 파일 편집까지 물리적으로 불가능해지는 것은 아니다. 서버가 필요한 작업과 로컬 작업을 구분한다.

근거 실습1 p.7-8

## 03 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). 중앙집중식 협업 방식이어도 각 Git 저장소의 분산 구조는 유지된다 (1점).

해설

저장소 기술 구조와 팀의 운영 워크플로는 구분한다.

근거 실습1 p.8, 12-13

# 01회 정답·해설

VCS·워크플로·터미널 | 2 / 5

## 04 | 연결형 · 내 점수 ___ / 2

정답 · 채점 기준

A: Integration-Manager (1점). B: Dictator and Lieutenant (1점).

해설

검토·취합 단계의 수와 역할을 보면 구분할 수 있다.

근거 실습1 p.14-15

## 05 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

④

해설

Git은 분산 시스템을 목표로 했다. 중앙 서버의 매번 승인을 필수로 하지 않는다.

근거 실습1 p.10

## 06 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

cd .. ; pwd (1점). 출력: /home/so/project (1점).

해설

cd ..는 부모로 이동하고 pwd는 현재 위치를 출력한다. 여기의 세미콜론은 명령 순서 표시다.

근거 실습1 p.34

# 01회 정답·해설

VCS·워크플로·터미널 | 3 / 5

## 07 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

/home/so/A (1점). 바로 이전 작업 디렉토리로 이동 (1점).

해설

cd -와 상위 디렉토리로 가는 cd ..를 혼동하지 않는다.

근거 실습1 p.34

## 08 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

mkdir -p exam/git/practice (1점). 없는 상위 디렉토리도 함께 생성 (1점).

해설

mkdir만으로 중간 경로가 없는 중첩 디렉토리를 모두 만들 수 있는 것은 아니다.

근거 실습1 p.35

## 09 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

①

해설

ls의 -a는 숨김 포함, -l은 상세 정보, -h는 크기 표시 형식을 조정한다.

근거 실습1 p.36-37

# 01회 정답·해설

VCS·워크플로·터미널 | 4 / 5

## 10 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). 터미널 화면을 정리할 뿐 파일·이력을 삭제하지 않는다 (1점).

해설

화면에서 안 보이는 것과 저장된 데이터 삭제는 다른 행위다.

근거 실습1 p.38

## 11 | 오류수정 · 내 점수 ___ / 2

정답 · 채점 기준

rm -r temp (1점). temp 및 그 아래 파일·디렉토리 삭제 (1점).

해설

재귀 삭제 범위를 이해하는 문제다. 실제 실행 전 대상 경로를 확인해야 한다.

근거 실습1 p.39

## 12 | 빈칸 · 내 점수 ___ / 2

정답 · 채점 기준

A: cd ~ (1점). B: exit (1점).

해설

폴더 이동과 세션 종료를 구분한다.

근거 실습1 p.34, 42

# 01회 정답·해설

VCS·워크플로·터미널 | 5 / 5

## 13 | 조건추론 · 내 점수 ___ / 2

정답 · 채점 기준

ls -a 또는 ls -la (1점). .으로 시작하는 숨김 항목이 기본 목록에서 생략될 수 있음 (1점).

해설

보이지 않음을 존재하지 않음과 혼동하지 않는다.

근거 실습1 p.36-37, 48-50

## 14 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git config --global user.name "SSW" (1점). 현재 사용자 계정의 기본 Git 설정 (1점).

해설

--global은 모든 컴퓨터 사용자에게 무조건 적용되는 뜻이 아니다. 저장소별 설정이 있으면 덮어쓸 수 있다.

근거 실습1 p.32

## 15 | 통합판단 · 내 점수 ___ / 2

정답 · 채점 기준

복제본의 저장소 이력으로 복구 가능 (1점). 다른 사람의 미커밋 파일·복제 후 서버에만 생긴 커밋 등은 없을 수 있음 (1점).

해설

'완전한 백업' 설명은 복제 당시 포함된 Git 데이터에 관한 것이다. 모든 개인 작업까지 자동 포함하지 않는다.

근거 실습1 p.8, 50-53

# 02회 문제

작업트리·스테이지·커밋 | 1 / 5

## 16 | 객관식 · 2점

처음 만드는 프로젝트 폴더를 Git 저장소로 초기화하는 명령은?
① git log
② git init
③ git switch
④ git commit

답 / 풀이

## 17 | 순서배열 · 2점

새 파일 a.txt를 첫 버전으로 기록한다. 저장소는 이미 있다.
A git commit -m "first" / B a.txt 작성·저장 / C git add a.txt
순서를 쓰시오.

답 / 풀이

## 18 | 결과추적 · 2점

기존 커밋의 a.txt는 A이다. 내용을 B로 저장하고 git add a.txt를 실행한 뒤, C로 다시 저장하고 git commit -m "next"를 했다. 커밋 내용과 작업트리 내용을 쓰시오.

답 / 풀이

# 02회 문제

작업트리·스테이지·커밋 | 2 / 5

## 19 | 조건판정 · 2점

앞 문제와 독립된 상황이다. 추적 중인 x.txt를 수정했지만 add하지 않았다. 다른 스테이징된 변경도 없다. git commit -m "x"만 실행하면 새 버전이 만들어지는가? 이유도 쓰시오.

답 / 풀이

## 20 | 결과추적 · 2점

추적 중인 a.txt를 수정하고, 새 b.txt를 작성했다. b.txt는 한 번도 add하지 않았다. git commit -am "update" 실행 시 어떤 변경이 기록되는가?

답 / 풀이

## 21 | 복수선택 · 2점

커밋 메시지 옵션 -m의 기능에 대해 옳은 것을 모두 고르시오.
① 커밋 메시지를 지정
② 미추적 파일을 자동 추가
③ 일반적으로 스테이징 내용을 기록하는 commit에 사용 가능
④ 원격에 자동 업로드

답 / 풀이

# 02회 문제

작업트리·스테이지·커밋 | 3 / 5

## 22 | 상태추론 · 2점

a.txt를 새로 만든 뒤 add했고, 커밋하기 전에 다시 수정했다. 같은 파일이 'Changes to be committed'와 'Changes not staged for commit'에 동시에 나타날 수 있는가? 이유도 쓰시오.

답 / 풀이

## 23 | 명령작성 · 2점

a.txt와 b.txt를 수정했지만 이번 버전에는 a.txt만 넣으려 한다. 스테이지는 비어 있다. 사용할 두 명령을 순서대로 쓰시오.

답 / 풀이

## 24 | 객관식 · 2점

커밋은 성공했는데 팀원의 컴퓨터에는 바뀐 내용이 없다. 가장 정확한 설명은?
① 커밋은 로컬 기록이며 공유는 별도
② commit은 언제나 전 세계 동기화
③ Git에 오류가 있다는 유일한 증거
④ 메시지가 짧으면 공유 불가

답 / 풀이

# 02회 문제

작업트리·스테이지·커밋 | 4 / 5

## 25 | 오류찾기 · 2점

다음 설명의 두 오류를 고쳐라. A “Modified는 반드시 한 번도 추적되지 않은 새 파일.” B “Staged는 이미 새 커밋에 영구 기록된 상태.”

답 / 풀이

## 26 | 빈칸 · 2점

현재 변경 상태를 확인하는 명령은 (A), 이미 만들어진 커밋 이력을 확인하는 명령은 (B)이다.

답 / 풀이

## 27 | 결과추적 · 2점

추적 중인 a.txt=A, b.txt=X. a를 B로 수정·add, b를 Y로 수정한 뒤 일반 commit을 했다. 새 커밋의 a와 b 내용은?

답 / 풀이

# 02회 문제

작업트리·스테이지·커밋 | 5 / 5

## 28 | OX·수정 · 2점

“git add는 파일 이름만 예약하므로 커밋 순간 디스크에 저장된 최신 내용이 무조건 들어간다.” O/X와 바른 설명을 쓰시오.

답 / 풀이

## 29 | 명령복구 · 2점

추적 파일의 수정본 B를 add한 뒤 최종본 C로 고쳤다. C를 이번 일반 커밋에 넣고 싶다. 추가로 할 명령과 그 이유를 쓰시오.

답 / 풀이

## 30 | 통합사례 · 2점

추적 파일 old.txt를 수정하고 새 new.txt를 만들었다. 두 파일만 커밋하려 한다. 스테이지는 비어 있고 다른 변경은 없다. add·commit 두 명령과 확인 명령 하나를 쓰시오.

답 / 풀이

# 02회 정답·해설

작업트리·스테이지·커밋 | 1 / 5

## 16 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

②

해설

init은 저장소를 만들고, log는 이력 조회, switch는 브랜치 전환, commit은 버전 기록이다.

근거 실습1 p.47-49, 67

## 17 | 순서배열 · 내 점수 ___ / 2

정답 · 채점 기준

B → C → A

해설

작업트리의 내용을 저장하고 스테이징한 뒤 커밋한다.

근거 실습1 p.50-53, 57-59

## 18 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

커밋: B (1점). 작업트리: C (1점).

해설

일반 commit은 마지막 add로 준비한 내용을 기록한다. add 뒤의 수정은 자동 반영되지 않는다.

근거 실습1 p.50-53, 58-59; 보완 T

# 02회 정답·해설

작업트리·스테이지·커밋 | 2 / 5

## 19 | 조건판정 · 내 점수 ___ / 2

정답 · 채점 기준

아니다 (1점). 일반 commit에 기록할 스테이징된 변경이 없다 (1점).

해설

이 문제는 -a나 파일 경로 지정 없이 일반 commit을 사용한다.

근거 실습1 p.11, 52-53

## 20 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

a.txt 변경 기록 (1점). b.txt는 미추적이므로 기록되지 않음 (1점).

해설

-a는 이미 추적 중인 파일의 수정·삭제를 대상으로 한다. 모든 새 파일을 자동 추가하지 않는다.

근거 실습1 p.57, 61; 보완 C

## 21 | 복수선택 · 내 점수 ___ / 2

정답 · 채점 기준

①, ③

해설

-m은 메시지 옵션이다. add나 원격 전송의 역할은 아니다.

근거 실습1 p.59, 61

# 02회 정답·해설

작업트리·스테이지·커밋 | 3 / 5

## 22 | 상태추론 · 내 점수 ___ / 2

정답 · 채점 기준

가능하다 (1점). 인덱스에는 add 시점 내용, 작업트리에는 이후 수정 내용이 있기 때문 (1점).

해설

파일 하나에도 커밋·스테이지·작업트리 버전이 서로 다를 수 있다.

근거 실습1 p.11, 50-53, 58; 보완 T

## 23 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git add a.txt (1점). git commit -m "update a" (1점).

해설

전체 추가나 commit -a를 쓰면 원치 않는 추적 파일 변경까지 들어갈 수 있다.

근거 실습1 p.52-53, 59

## 24 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

①

해설

Git의 분산 구조에서는 로컬 버전 기록과 원격 공유를 구분한다.

근거 실습1 p.8, 12-14, 53

# 02회 정답·해설

작업트리·스테이지·커밋 | 4 / 5

## 25 | 오류찾기 · 내 점수 ___ / 2

정답 · 채점 기준

A: 추적 파일도 수정되면 Modified가 됨 (1점). B: Staged는 다음 커밋에 준비된 상태 (1점).

해설

Untracked와 Modified, Staged와 Committed를 각각 구분한다.

근거 실습1 p.11, 50, 57-59

## 26 | 빈칸 · 내 점수 ___ / 2

정답 · 채점 기준

A: git status (1점). B: git log (1점).

해설

status와 log는 각각 현재 작업 상태와 기록된 이력이라는 다른 질문에 답한다.

근거 실습1 p.56, 60, 62

## 27 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

a=B (1점). b=X (1점).

해설

b의 Y는 작업트리에만 남는다. 스테이징하지 않은 기존 파일도 커밋에서 사라지는 것이 아니라 이전 내용이 유지된다.

근거 실습1 p.50-53, 58-59

# 02회 정답·해설

작업트리·스테이지·커밋 | 5 / 5

## 28 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). add 시점의 내용을 스테이징하며 이후 수정은 다시 add해야 한다 (1점).

해설

스테이징 영역을 단순 파일 목록으로 생각하면 중간 수정 문제를 틀리기 쉽다.

근거 실습1 p.52-53, 58; 보완 T

## 29 | 명령복구 · 내 점수 ___ / 2

정답 · 채점 기준

git add 해당파일 (1점). 인덱스를 최종본 C로 다시 갱신해야 함 (1점).

해설

다시 add하면 같은 파일의 준비된 내용이 현재 내용으로 바뀐다.

근거 실습1 p.52-53, 58-59

## 30 | 통합사례 · 내 점수 ___ / 2

정답 · 채점 기준

git add old.txt new.txt ; git commit -m "update" (1점). git status 또는 git log (1점).

해설

새 파일은 명시적으로 추가해야 한다. status는 남은 변경, log는 만들어진 기록을 확인한다.

근거 실습1 p.56-62

# 03회 문제

브랜치 포인터와 이력 범위 | 1 / 5

## 31 | 객관식 · 2점

main에서 git branch feature를 실행했다. 가장 정확한 결과는?
① feature 생성과 동시에 전환
② feature만 생성하고 현재 브랜치는 main
③ main 삭제
④ 자동 병합

답 / 풀이

## 32 | 명령작성 · 2점

현재 브랜치에서 kakao를 만들고, 그 브랜치로 전환하라. 강의의 두 명령을 순서대로 쓰시오.

답 / 풀이

## 33 | 포인터추적 · 2점

커밋 A를 main이 가리킨다. 현재 main에서 git branch test로 새 브랜치를 만들었고 전환 없이 새 커밋 B를 만들었다. main과 test가 가리키는 커밋은?

답 / 풀이

# 03회 문제

브랜치 포인터와 이력 범위 | 2 / 5

## 34 | 연결형 · 2점

강의의 브랜치 역할을 연결하라. A 긴급 운영 버그 / B 새 기능 / C 다음 버전 통합 / D 출시 준비.
보기: hotfix, feature, develop, release

답 / 풀이

## 35 | OX·수정 · 2점

“브랜치를 10개 만들면 매번 프로젝트 전체 폴더 10개를 물리적으로 새로 복사해야 한다.” O/X와 핵심 개념을 쓰시오.

답 / 풀이

## 36 | 집합추론 · 2점

A→B는 공통 이력, main은 B→C, kakao는 B→D→E이다. git log main..kakao에 포함되는 커밋 이름만 쓰시오.

답 / 풀이

# 03회 문제

브랜치 포인터와 이력 범위 | 3 / 5

## 37 | 집합추론 · 2점

A→B 공통 뒤 main에 C→F, kakao에 D→E가 있다. git log kakao..main에 포함되는 커밋을 쓰시오.

답 / 풀이

## 38 | 명령작성 · 2점

naver에는 없고 main에서만 도달 가능한 커밋을 보고 싶다. 명령과 앞·뒤 브랜치의 의미를 쓰시오.

답 / 풀이

## 39 | 객관식 · 2점

브랜치 그래프 화면에서 kakao가 main보다 위에 보인다. 확실히 말할 수 있는 것은?
① kakao가 반드시 제품 출시용
② main에 kakao가 병합됨
③ 화면의 위아래만으로 조상·병합 관계를 확정할 수 없음
④ main이 손상됨

답 / 풀이

# 03회 문제

브랜치 포인터와 이력 범위 | 4 / 5

## 40 | 포인터추적 · 2점

A에서 main·feature가 갈라졌다. feature로 전환해 B, C를 커밋했다. main에는 작업이 없다. main으로 전환하면 두 브랜치 포인터는 어디에 있는가?

답 / 풀이

## 41 | 조건추론 · 2점

git log main..test가 비어 있다. “test가 존재하지 않는다”는 결론이 반드시 맞는가? 가능한 정상 상황 두 가지를 쓰시오.

답 / 풀이

## 42 | 빈칸 · 2점

git log --oneline --graph에서 --oneline은 (A), --graph는 (B)에 도움을 준다.

답 / 풀이

# 03회 문제

브랜치 포인터와 이력 범위 | 5 / 5

## 43 | 오류수정 · 2점

main에서 기능 작업을 시작하기 전 git branch feature만 실행했다. 이후 main을 건드리지 않고 기능 커밋을 만들려면, 수정·커밋 전에 무엇을 해야 하며 왜인가?

답 / 풀이

## 44 | 역추론 · 2점

서로 다른 이름 main·test가 정상 존재한다. main..test와 test..main이 모두 비어 있고, 각 브랜치는 일반 커밋 하나를 끝으로 가리킨다. 두 끝 커밋의 관계와 파일 상태에 대해 무엇을 결론낼 수 있는가?

답 / 풀이

## 45 | 통합설계 · 2점

팀원 3명이 검색·추천·코스를 개발한다. main을 안정적으로 두기 위한 브랜치 구성과 통합 시점 원칙을 제안하라.

답 / 풀이

# 03회 정답·해설

브랜치 포인터와 이력 범위 | 1 / 5

## 31 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

②

해설

브랜치 생성과 전환은 별개다.

근거 실습3 p.14-15

## 32 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git branch kakao (1점). git switch kakao (1점).

해설

생성만 하면 새 작업이 기존 브랜치에 기록될 수 있다.

근거 실습3 p.14-15

## 33 | 포인터추적 · 내 점수 ___ / 2

정답 · 채점 기준

main=B (1점). test=A (1점).

해설

새 커밋은 현재 브랜치를 전진시킨다. 생성된 다른 브랜치가 자동으로 따라오지 않는다.

근거 실습3 p.6-8, 14-15

# 03회 정답·해설

브랜치 포인터와 이력 범위 | 2 / 5

## 34 | 연결형 · 내 점수 ___ / 2

정답 · 채점 기준

A-hotfix, B-feature (각 0.5점). C-develop, D-release (각 0.5점).

해설

이름은 팀 규칙으로 쓰는 역할 구분이다. Git이 이름만 보고 자동 처리하는 기능은 아니다.

근거 실습3 p.5

## 35 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). Git 브랜치는 커밋을 가리키는 포인터·독립 작업 흐름이다 (1점).

해설

강의의 복사 비유를 실제 디렉토리 복제와 동일시하지 않는다.

근거 실습3 p.4, 6

## 36 | 집합추론 · 내 점수 ___ / 2

정답 · 채점 기준

D, E

해설

main에서 도달하지 못하지만 kakao에서는 도달하는 커밋이다. 출력 순서는 채점하지 않는다.

근거 실습3 p.17

# 03회 정답·해설

브랜치 포인터와 이력 범위 | 3 / 5

## 37 | 집합추론 · 내 점수 ___ / 2

정답 · 채점 기준

C, F

해설

범위 좌우를 바꾸면 차집합의 방향이 바뀐다.

근거 실습3 p.17

## 38 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git log naver..main (1점). main의 이력에서 naver의 이력을 제외 (1점).

해설

두 점 범위는 파일 내용 비교 명령이 아니다.

근거 실습3 p.17

## 39 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

③

해설

그래프의 연결선과 커밋 관계를 읽어야 한다. 강의도 순서·색은 중요하지 않다고 한다.

근거 실습3 p.18-19

# 03회 정답·해설

브랜치 포인터와 이력 범위 | 4 / 5

## 40 | 포인터추적 · 내 점수 ___ / 2

정답 · 채점 기준

main=A (1점). feature=C (1점).

해설

전환은 현재 작업 대상을 바꾸며 다른 브랜치를 병합하는 동작이 아니다.

근거 실습3 p.7-9, 15

## 41 | 조건추론 · 내 점수 ___ / 2

정답 · 채점 기준

두 브랜치가 같은 커밋을 가리킴 (1점). test가 main의 조상이며 고유 커밋이 없음 (1점).

해설

test의 모든 이력이 main에 포함되면 차집합은 비어 있다.

근거 실습3 p.17, 26

## 42 | 빈칸 · 내 점수 ___ / 2

정답 · 채점 기준

A: 커밋을 한 줄로 요약 (1점). B: 커밋 이력의 연결 관계를 그래프로 표시 (1점).

해설

표시 옵션이지 새로운 커밋이나 브랜치를 만드는 옵션이 아니다.

근거 실습3 p.18

# 03회 정답·해설

브랜치 포인터와 이력 범위 | 5 / 5

## 43 | 오류수정 · 내 점수 ___ / 2

정답 · 채점 기준

git switch feature (1점). 새 커밋이 feature에 기록되도록 현재 브랜치를 바꿔야 함 (1점).

해설

이 문제는 아직 수정·커밋 전이다. 이미 만든 커밋을 되돌리는 명령은 요구하지 않는다.

근거 실습3 p.14-15

## 44 | 역추론 · 내 점수 ___ / 2

정답 · 채점 기준

같은 끝 커밋을 가리킨다 (1점). 따라서 커밋에 기록된 프로젝트 상태도 같다 (1점).

해설

정상 Git 이력은 순환하지 않는다. 서로의 도달 가능한 이력이 동일한 두 끝은 같은 커밋이다. 미커밋 작업 상태는 별개다.

근거 실습3 p.6, 17 기반 응용

## 45 | 통합설계 · 내 점수 ___ / 2

정답 · 채점 기준

기능별 feature 브랜치로 분리 (1점). 각 기능 검토·시험 후 main에 병합 (1점).

해설

브랜치 수만 늘리는 것이 아니라 작업 경계와 통합 기준을 정한다.

근거 실습3 p.4-9

# 04회 문제

병합 방향·충돌·빨리감기 | 1 / 5

## 46 | 객관식 · 2점

이력은 A→B→C, main=A, test=C이다. 깨끗한 main에서 기본 git merge test를 실행한다. 결과는?

① fast-forward
② 반드시 충돌
③ 이미 최신
④ main 삭제

답 / 풀이

## 47 | 포인터추적 · 2점

A→B→C, main=C, naver=B이다. main에서 git merge naver를 했다. 메시지의 의미와 main의 위치를 쓰시오.

답 / 풀이

## 48 | 명령작성 · 2점

main=C, naver=B이며 B는 C의 조상이다. naver에 main의 변경을 가져오려 한다. 두 명령을 쓰시오.

답 / 풀이

# 04회 문제

병합 방향·충돌·빨리감기 | 2 / 5

## 49 | OX·수정 · 2점

“git merge test는 어느 브랜치에서 실행해도 main을 갱신한다.” O/X와 바른 기준을 쓰시오.

답 / 풀이

## 50 | 조건판정 · 2점

A 이후 main은 B, feature는 C로 각각 진행했다. 변경 부분은 독립적이다. 기본 병합은 fast-forward인가? 충돌은 필수인가?

답 / 풀이

## 51 | 결과추적 · 2점

공통 파일의 line1='red', line2='small'. main은 line1을 blue로, test는 같은 line1을 green으로 바꿨다. 충돌이 난다고 가정한다. 누가 결정하며, 자동으로 최신 시각의 값이 이기는가?

답 / 풀이

# 04회 문제

병합 방향·충돌·빨리감기 | 3 / 5

## 52 | 객관식 · 2점

main은 a.txt만 수정, test는 서로 독립적인 b.txt만 수정했다. 공통 조상 이후 둘 다 커밋했다. 가장 적절한 예상은?
① 분기했으므로 무조건 충돌
② 내용 충돌 없이 자동 병합 가능
③ 파일 이름이 다르면 이력 손실
④ 병합 금지

답 / 풀이

## 53 | 사례서술 · 2점

충돌이 난 뒤 파일의 갈등 부분을 눈으로만 읽고 “해결 완료”라고 했다. 실제 마무리에 필요한 작업 두 단계를 쓰시오.

답 / 풀이

## 54 | 포인터추적 · 2점

A→B→C, main=A, test=C. main에서 기본 fast-forward 병합 후 test의 위치는? main의 위치도 쓰시오.

답 / 풀이

# 04회 문제

병합 방향·충돌·빨리감기 | 4 / 5

## 55 | 집합추론 · 2점

main이 test를 성공적으로 병합했고 그 뒤 양쪽에 새 커밋은 없다. git log main..test의 결과와 이유를 쓰시오.

답 / 풀이

## 56 | 오류진단 · 2점

naver를 최신으로 만들려고 main에서 git merge naver를 실행했더니 Already up to date가 나왔다. main이 더 앞선 상황이다. 실수와 올바른 방향을 쓰시오.

답 / 풀이

## 57 | 빈칸 · 2점

Fast-forward는 현재 브랜치 끝이 상대 브랜치 끝의 (A)일 때 가능하며, 기본 동작에서는 별도 병합 커밋 대신 (B)를 이동한다.

답 / 풀이

# 04회 문제

병합 방향·충돌·빨리감기 | 5 / 5

## 58 | 조건비교 · 2점

A: main=A, test=B이고 A→B. B: main=C, test=D이고 A에서 C와 D가 갈라짐. 각 상황의 기본 병합 방식 가능성을 쓰시오.

답 / 풀이

## 59 | 검증설계 · 2점

충돌 없이 병합됐는데 실행하면 오류가 난다. 이 현상이 가능한 이유와 다음 행동을 쓰시오.

답 / 풀이

## 60 | 통합추적 · 2점

A→B→C, main=C, test=A. test로 전환해 git merge main, 이어서 main으로 전환해 git merge test를 했다. 첫째와 둘째 병합 결과를 쓰시오.

답 / 풀이

# 04회 정답·해설

병합 방향·충돌·빨리감기 | 1 / 5

## 46 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

①

해설

main이 test의 조상이므로 main 포인터를 C로 이동하면 된다.

근거 실습3 p.9-10, 28

## 47 | 포인터추적 · 내 점수 ___ / 2

정답 · 채점 기준

Already up to date: naver 이력이 이미 포함됨 (1점). main=C 유지 (1점).

해설

병합은 브랜치 이름을 덮어쓰는 동작이 아니다. 뒤의 브랜치를 병합해도 과거로 돌아가지 않는다.

근거 실습3 p.24-26

## 48 | 명령작성 · 내 점수 ___ / 2

정답 · 채점 기준

git switch naver (1점). git merge main (1점).

해설

받는 브랜치로 전환한 다음 가져올 브랜치를 지정한다.

근거 실습3 p.27-28

# 04회 정답·해설

병합 방향·충돌·빨리감기 | 2 / 5

## 49 | OX·수정 · 내 점수 ___ / 2

정답 · 채점 기준

X (1점). 현재 브랜치에 test의 이력을 병합한다 (1점).

해설

명령 인수는 가져올 대상이며 받는 쪽은 현재 브랜치다.

근거 실습3 p.24-28

## 50 | 조건판정 · 내 점수 ___ / 2

정답 · 채점 기준

fast-forward 아님 (1점). 독립 수정이면 충돌 없이 자동 병합 가능 (1점).

해설

이력 분기와 내용 충돌은 다른 문제다. 기본 병합은 두 이력을 연결하는 병합 커밋을 만들 수 있다.

근거 실습3 p.11-12

## 51 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

개발자가 의도에 맞게 해결 (1점). 최신 시각의 값이 자동 승리하는 규칙은 아님 (1점).

해설

충돌은 내용과 의도에 대한 판단이 필요하다.

근거 실습3 p.11-12

# 04회 정답·해설

병합 방향·충돌·빨리감기 | 3 / 5

## 52 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

②

해설

분기 자체와 같은 부분 수정에 따른 충돌을 구분한다.

근거 실습3 p.11-12

## 53 | 사례서술 · 내 점수 ___ / 2

정답 · 채점 기준

원하는 최종 내용으로 편집하고 충돌 표시 제거 (1점). 해결 내용을 add한 뒤 병합 커밋 완료 (1점).

해설

파일 내용의 해결과 Git에 해결 상태를 기록하는 절차가 모두 필요하다.

근거 실습3 p.12; 보완 M

## 54 | 포인터추적 · 내 점수 ___ / 2

정답 · 채점 기준

test=C 유지 (1점). main=C (1점).

해설

병합했다고 가져온 브랜치가 자동 삭제되거나 다른 곳으로 이동하지 않는다.

근거 실습3 p.9-10, 28

# 04회 정답·해설

병합 방향·충돌·빨리감기 | 4 / 5

## 55 | 집합추론 · 내 점수 ___ / 2

정답 · 채점 기준

출력할 커밋 없음 (1점). test에서 도달하는 이력이 main에 모두 포함됨 (1점).

해설

파일 이름이 같아서가 아니라 커밋 포함 관계 때문이다.

근거 실습3 p.17, 26-28

## 56 | 오류진단 · 내 점수 ___ / 2

정답 · 채점 기준

받는 브랜치를 잘못 선택 (1점). naver로 전환해 main을 병합 (1점).

해설

강의의 main·naver 예제와 같은 병합 방향 문제다.

근거 실습3 p.24-28

## 57 | 빈칸 · 내 점수 ___ / 2

정답 · 채점 기준

A: 조상 (1점). B: 현재 브랜치 포인터 (1점).

해설

특별한 --no-ff 설정이 없는 기본 병합을 전제한다.

근거 실습3 p.10, 28

# 04회 정답·해설

병합 방향·충돌·빨리감기 | 5 / 5

## 58 | 조건비교 · 내 점수 ___ / 2

정답 · 채점 기준

A: fast-forward 가능 (1점). B: fast-forward 불가, 이력 결합 병합 필요 (1점).

해설

코드 충돌 여부는 이력 모양만으로 확정할 수 없다.

근거 실습3 p.10-12, 28

## 59 | 검증설계 · 내 점수 ___ / 2

정답 · 채점 기준

Git의 텍스트 병합 성공이 기능의 논리적 호환성을 보장하지 않음 (1점). 통합 동작·관련 기능을 실행해 시험하고 수정 (1점).

해설

서로 다른 줄에서 함수 이름과 호출 방식이 어긋나는 등의 오류가 가능하다.

근거 실습3 p.9, 12 기반 응용

## 60 | 통합추적 · 내 점수 ___ / 2

정답 · 채점 기준

첫째: test가 C로 fast-forward (1점). 둘째: 이미 같은 C이므로 Already up to date (1점).

해설

첫 병합 후 상태를 갱신해서 두 번째를 판단해야 한다.

근거 실습3 p.24-28

# 05회 문제

복합 상태 추적·역추론 | 1 / 5

## 61 | 결과추적 · 2점

main·f가 A를 가리킨다. f에서 B를 커밋, main으로 전환해 C를 커밋했다. f에는 없고 main에만 있는 커밋 집합과 명령을 쓰시오.

답 / 풀이

## 62 | 상태추론 · 2점

main=A, f=A에서 f로 전환했다. 추적 파일을 B 내용으로 수정·add한 뒤 C 내용으로 다시 수정하고 일반 commit K를 했다. K의 파일 내용과 main의 포인터는?

답 / 풀이

## 63 | 객관식 · 2점

브랜치 이름을 main에서 release라고 부르기로 했다는 이유만으로 Git이 자동 수행하는 것은?

① 서버 배포
② 시험·승인
③ 라이선스 고지
④ 제시된 동작 중 없음

답 / 풀이

# 05회 문제

복합 상태 추적·역추론 | 2 / 5

## 64 | 통합추론 · 2점

기본 병합으로 main에 f를 합쳐 병합 커밋 M이 만들어졌다. M의 두 부모는 main의 기존 B와 f의 C다. f는 C에 그대로 있다. git log f..main에 M이 포함되는가? 이유도 쓰시오.

답 / 풀이

## 65 | 반례제시 · 2점

“서로 다른 브랜치이면 언제나 파일 내용이 다르다.” 반례와 브랜치의 본질을 쓰시오.

답 / 풀이

## 66 | 조건판정 · 2점

서로 독립적으로 만든 서로 다른 커밋 B와 C의 파일 내용은 우연히 같다. A에서 분기해 main=B, f=C다. 내용이 같으니 main..f가 비는가? 이유도 쓰시오.

답 / 풀이

# 05회 문제

복합 상태 추적·역추론 | 3 / 5

## 67 | 명령설계 · 2점

main은 A, f는 A→B→C다. main에 f를 기본 병합하고 main에 없는 f 고유 커밋이 남았는지 확인하려 한다. 현재 f이다. 명령 세 개를 쓰시오.

답 / 풀이

## 68 | 결과추적 · 2점

추적 파일 a=A, 새 b=B이다. a를 C로 수정한 후 git commit -am "one"을 실행했다. 이어 git add b와 일반 commit "two"를 성공시켰다. 첫째·둘째 커밋의 파일 상태를 쓰시오.

답 / 풀이

## 69 | 오류찾기 · 2점

“스냅샷이면 매번 모든 파일을 별도 복제해야 하며, 브랜치도 반드시 폴더 복사본이다.” 두 오류를 고쳐라.

답 / 풀이

# 05회 문제

복합 상태 추적·역추론 | 4 / 5

## 70 | 조건추론 · 2점

main=A→B, f=A→C였고 main에 f를 병합해 M을 만들었다. 이후 f에서 D를 추가했다. git log main..f에서 새로 확인되는 커밋은 무엇이며 왜인가?

답 / 풀이

## 71 | 사례서술 · 2점

함수 정의를 수정한 브랜치와 호출부를 수정한 브랜치를 각각 단독 시험했다. 병합은 충돌 없이 끝났다. 통합 시험이 필요한 이유와 시험 항목을 쓰시오.

답 / 풀이

## 72 | 복합추적 · 2점

A에서 main·f가 출발했다. f에서 B를 만들고 main에 기본 병합한 뒤, main에서 C를 만들었다. 현재 f에서 git merge main을 한다. f의 최종 위치와 별도 병합 커밋 생성 여부를 쓰시오.

답 / 풀이

# 05회 문제

복합 상태 추적·역추론 | 5 / 5

## 73 | 집합비교 · 2점

main 이력={A,B,C,M}, f 이력={A,C}, g 이력={A,D}. M은 B와 C의 병합 커밋이다. main..f와 main..g의 결과를 각각 쓰시오.

답 / 풀이

## 74 | 복수선택 · 2점

커밋 직전 검사로 타당한 것을 모두 고르시오.
① 현재 브랜치 확인
② status로 준비된 변경 확인
③ 새 파일이 필요한 경우 add 여부 확인
④ 화면을 clear했으니 모든 변경이 저장됐다고 판단

답 / 풀이

## 75 | 최종설계 · 2점

팀원이 “파일을 저장했으니 버전 기록과 팀 통합까지 끝났다”고 한다. 로컬 버전 기록과 팀 통합을 각각 어떤 단계로 보완할지 쓰시오.

답 / 풀이

# 05회 정답·해설

복합 상태 추적·역추론 | 1 / 5

## 61 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

집합: C (1점). git log f..main (1점).

해설

B와 C는 서로 다른 분기의 끝이다. A는 공통 조상이라 제외된다.

근거 실습3 p.11, 17

## 62 | 상태추론 · 내 점수 ___ / 2

정답 · 채점 기준

K의 내용=B (1점). main=A 유지 (1점).

해설

스테이지 규칙과 현재 브랜치 포인터 규칙을 동시에 적용한다. 작업트리에는 C가 남는다.

근거 실습1 p.52-53; 실습3 p.7-8

## 63 | 객관식 · 내 점수 ___ / 2

정답 · 채점 기준

④

해설

브랜치 이름의 관례는 운영 정책이다. Git이 그 단어의 업무를 자동 수행하지 않는다.

근거 실습3 p.5 기반 응용

# 05회 정답·해설

복합 상태 추적·역추론 | 2 / 5

## 64 | 통합추론 · 내 점수 ___ / 2

정답 · 채점 기준

포함된다 (1점). M은 main에서 도달 가능하지만 f의 C에서는 도달 불가 (1점).

해설

부모 방향으로 이력을 읽는다. f의 내용이 합쳐졌다는 것과 f가 M으로 이동했다는 것은 다르다.

근거 실습3 p.11-12, 17 기반 응용

## 65 | 반례제시 · 내 점수 ___ / 2

정답 · 채점 기준

같은 커밋에서 새 브랜치를 만든 직후 내용이 같음 (1점). 브랜치는 커밋을 가리키는 포인터 (1점).

해설

이름의 차이가 파일 내용의 차이를 보장하지 않는다.

근거 실습3 p.6-8, 14

## 66 | 조건판정 · 내 점수 ___ / 2

정답 · 채점 기준

아니다, C가 포함될 수 있다 (1점). log 범위는 파일 내용이 아니라 커밋 도달 관계를 비교 (1점).

해설

동일한 내용과 동일한 커밋 이력은 다른 개념이다. B·C는 문제에서 서로 다른 커밋으로 주어졌다.

근거 실습3 p.6, 17 기반 응용

# 05회 정답·해설

복합 상태 추적·역추론 | 3 / 5

## 67 | 명령설계 · 내 점수 ___ / 2

정답 · 채점 기준

git switch main ; git merge f (1점). git log main..f (1점).

해설

마지막 출력은 비어 있어야 한다. 작업트리는 깨끗하며 기본 병합이 성공한다고 가정한다.

근거 실습3 p.15, 17, 28

## 68 | 결과추적 · 내 점수 ___ / 2

정답 · 채점 기준

첫째: a=C, b 미포함 (1점). 둘째: a=C, b=B (1점).

해설

두 번째 커밋은 b만 가진 폴더가 아니라 a의 기존 기록도 포함하는 프로젝트 스냅샷이다.

근거 실습1 p.50-53, 61; 보완 C

## 69 | 오류찾기 · 내 점수 ___ / 2

정답 · 채점 기준

스냅샷은 상태의 논리 모델이며 변경 없는 파일 데이터를 재사용 가능 (1점). 브랜치는 커밋 포인터 (1점).

해설

강의 실습3 p.6의 델타 설명은 논리적 스냅샷 모델과 저장 최적화를 구분해 읽는다.

근거 실습3 p.6; 보완 T

# 05회 정답·해설

복합 상태 추적·역추론 | 4 / 5

## 70 | 조건추론 · 내 점수 ___ / 2

정답 · 채점 기준

D (1점). C와 A는 M의 이력에 포함됐지만 D는 아직 main에 없음 (1점).

해설

병합 시점 이후에 생긴 변경은 자동으로 다른 브랜치에 따라가지 않는다.

근거 실습3 p.11-12, 17 기반 응용

## 71 | 사례서술 · 내 점수 ___ / 2

정답 · 채점 기준

각 브랜치의 단독 성공이 결합 후 인터페이스 일치를 보장하지 않음 (1점). 수정된 호출 경로·인자·반환값을 실제 실행해 확인 (1점).

해설

텍스트 충돌 탐지와 프로그램 의미의 검증은 다르다.

근거 실습3 p.9, 12 기반 응용

## 72 | 복합추적 · 내 점수 ___ / 2

정답 · 채점 기준

f=C (1점). fast-forward이므로 별도 병합 커밋 없음 (1점).

해설

첫 병합에서 main=B가 되고, main만 C로 진행한다. f=B는 C의 조상이므로 다시 포인터를 전진시킨다.

근거 실습3 p.10, 24-28 기반 응용

# 05회 정답·해설

복합 상태 추적·역추론 | 5 / 5

## 73 | 집합비교 · 내 점수 ___ / 2

정답 · 채점 기준

main..f: 없음 (1점). main..g: D (1점).

해설

두 점 표기는 뒤 집합에서 앞 집합을 뺀다. 배열 순서는 채점하지 않는다.

근거 실습3 p.17 기반 응용

## 74 | 복수선택 · 내 점수 ___ / 2

정답 · 채점 기준

①, ②, ③

해설

브랜치·스테이지·미추적 파일을 함께 확인해야 의도한 버전을 만들 수 있다.

근거 실습1 p.38, 56-59; 실습3 p.14-15

## 75 | 최종설계 · 내 점수 ___ / 2

정답 · 채점 기준

필요한 파일 add 후 commit으로 로컬 버전 기록 (1점). 변경을 공유하고 대상 브랜치에서 검토·병합·통합 시험 (1점).

해설

디스크 저장, Git 기록, 팀 공유와 통합은 서로 다른 단계다.

근거 실습1 p.12-14, 50-53; 실습3 p.9-12

# 근거 안내 · 채점 보완

정답의 약칭은 아래 첨부 자료를 뜻합니다. 페이지는 PDF의 첫 페이지를 1로 센 번호이며, 표지와 목차도 포함합니다.

### 실습1

20260911[오픈소스SW의_이해]CHAPTER01_실습.pdf

### 실습3

20260917[오픈소스SW의_이해]CHAPTER03_실습.pdf

---

### 명령·그래프 전제

특별한 설명이 없으면 일반 로컬 저장소, 깨끗한 작업트리, 정상적인 저장·명령 실행을 전제합니다. merge는 기본 설정이며 --no-ff 등의 강제 옵션은 없습니다. A→B는 B의 부모가 A라는 뜻입니다.

### 스냅샷 보완 T

실습3 p.6의 "마지막 스냅샷만 완전 저장, 이전은 델타"라는 문장은 Git의 논리 모델로 그대로 외우지 않습니다. 각 커밋은 프로젝트 스냅샷을 가리키며 변경 없는 파일을 재사용할 수 있습니다.

### 스테이지·-a 보완 C

add는 그 시점 내용을 준비합니다. 이후 수정은 다시 add해야 일반 commit에 반영됩니다. commit -a는 미추적 새 파일을 자동 추가하지 않습니다. --global은 현재 사용자 계정의 기본 설정입니다.

### 충돌 해결 보완 M

충돌 내용을 편집한 뒤 add로 해결 상태를 기록하고 병합 커밋을 완료합니다. 병합 성공과 프로그램 기능 시험 성공은 별개입니다.

### 공식 보완 자료

Git Book, What is Git?: https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F
git-commit: https://git-scm.com/docs/git-commit
Basic Branching and Merging: https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging